order based on the column '100' (which contains the number of centuries scored by each batsman) and stores the details of the top 10 century scorers into a CSV file called *Highest100.csv*. This file will also be stored in the working directory of JupyterLab. Finally, Line 7 extracts details of columns 5 and 7 (total runs scored and average, respectively) and generates a scatter plot. Figure 2 shows this scatter plot generated by the program on execution.

```
1  from numpy import random
2  import matplotlib.pyplot as plt
3  import seaborn as sns
4
5  binom=random.binomial(n=1000, p=0.01, size=1000)
6  sns.histplot(data=binom, kde=True, color='r')
7  po=random.poisson(lam=10, size=1000)
8  sns.histplot(data=po, kde=True, color='g')
9  exp=random.exponential(size=1000)
10 sns.histplot(data=exp, kde=True, color='b')
11 plt.show()
```

Figure 4: Program for probability distributions

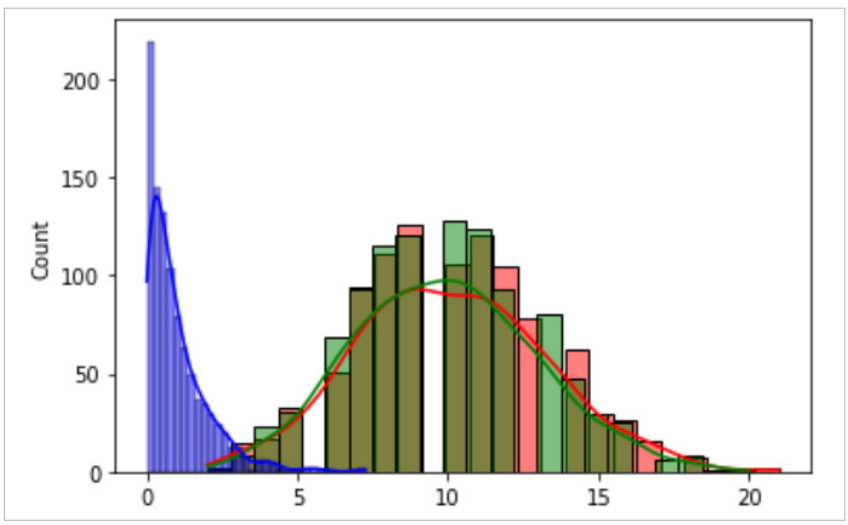

Figure 5: Plots of probability distributions

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  a = 10
5  b = 1
6  x = np.linspace(0,1,100)
7  y = a*x+b+np.random.randn(len(x))
8  plt.scatter(x, y, color = "r", marker = "x")
9  p = np.polyfit(x, y,1)
10 y_l = np.polyval(p,x)
11 plt.plot(x,y_l,color = "b",)
12 plt.show()
```

Figure 6: Linear regression with NumPy

Now add the following line of code at the end of the program and run it again.

```
sns.kdeplot(data=df.iloc[:, [5, 7]].
head(50), x='Ave', y='Runs')
```

This draws a KDE plot, in addition to the scatter plot, of the data obtained from columns 5 and 7. The function *kdeplot( )* gives a Kernel Distribution Estimation Plot, which depicts the probability density function of the continuous or non-parametric data variables. This definition may not give you any idea about the actual operation that will be performed by the function *kdeplot( )*. Figure 3 shows the KDE plot and scatter plot being drawn on a single image. From this figure, we can observe that data points drawn by the scatter plot are grouped into clusters by the KDE plot. Notice that there are a lot of other plotting functions also provided by seaborn. In the program we have discussed now, replace Line 7 with the lines of code (one line at a time) shown below, and execute the program again. You will see plots with varying styles. Explore the other plotting functions offered by seaborn and choose the one that suits your need the most.

```
sns.histplot(data=df.iloc[:, [5, 7]].
head(50), x='Ave', y='Runs')
```

```
sns.rugplot(data=df.iloc[:, [5, 7]].
head(50), x='Ave', y='Runs')
```

More on probability

Now, let us learn some more topics in probability. In one of the previous articles in this series, we saw how a normal distribution can be used to model real world scenarios. But normal distribution is just one among the many important probability distributions. Consider the program shown in Figure 4, which plots three probability distributions.